



Production RAG

AICB-P2T3 · Ngày 18 · Từ Demo 60% Đến Production 85%+

Trần Quang Thiện

[linkedin.com/in/thientquang](https://www.linkedin.com/in/thientquang)

VinUniversity · Phase 2 · Track 3 · Tuần 4 · 2026

HÃY SUY NGHĨ...

*“RAG pipeline demo chạy ngon, nhưng
lên production accuracy chỉ 60% — in-
gestion hay retrieval đang giết bạn?”*

*Day 07–08 dạy RAG chạy được. Hôm
nay: làm RAG đáng tin ở production.”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

- **01 · Map & Compass** — vì sao basic RAG hỏng, và **bản đồ** để biết sửa ở đâu.
- **02 · Offline — ingest** (*làm 1 lần · đặt trần*) — chuẩn bị dữ liệu để **đặt trần chất lượng** trước khi index.
- **03 · Online — retrieval** (*mỗi query · theo vấn đề*) — truy vấn từng query: **sửa theo vấn đề** để chunk đúng lên top.
- **04 · Close the Loop** — đo bằng RAGAS, tra ngược Error Tree, và biết **giới hạn**.

Hôm nay = **chất lượng ingestion + retrieval**. GraphRAG (Day 19), Multi-Agent (Day 20/23), RAGAS sâu & guardrails (Day 24), caching & reliability (Day 25), HITL (Day 27) học riêng.

- **Chẩn đoán** pipeline hỏng — Error Tree, không đoán mò
- **OFFLINE**: hierarchical/late chunk, contextual, enrichment
- **ONLINE retrieval**: vượt hybrid+RRF — late interaction, learned sparse, fusion
- **ONLINE rerank**: chọn theo chất lượng / chi phí / ngôn ngữ
- **ONLINE control flow**: HyDE, Multi-Query, CRAG theo vấn đề
- **RAGAS** + diagnostic tree → sửa đúng tầng
- **Giới hạn** embedding — phần nào để dành ngày sau

`src/` (5 module hoàn chỉnh) + `analysis/` (RAGAS report + failure analysis)

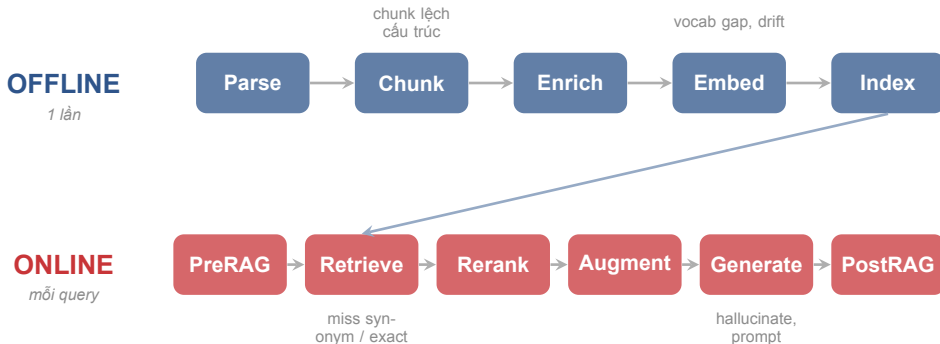
- `m1_chunking.py` — hierarchical chunking (parent 2048 / child 256)
- `m2_search.py` — hybrid search (BM25 + Dense + RRF)
- `m3_rerank.py` — reranking top-20 → top-3
- `m4_eval.py` — RAGAS end-to-end (4 metrics)
- `m5_enrichment.py` — enrichment pipeline (summarize, HyQA, contextual prepend, metadata)
- `analysis/` — bottom-5 failure analysis map vào Error Tree

PHẦN 01

Map & Compass

Trước khi sửa: bản đồ pipeline (OFFLINE + ONLINE), la bàn Error Tree, và ca chạy TaiLieuBot xuyên suốt cả ngày

RAG = OFFLINE + ONLINE, Mỗi Bước Đầu Có Thể Hỏng



Lần lượt mỗi sinh viên chọn **1 bước** trong pipeline và nói trong **1 phút**: bước đó *làm gì* và *có thể hỏng thế nào*. Mục tiêu — cả lớp cùng nắm bản đồ trước khi đi sâu Phần 02–03.

OFFLINE (1 lần): Parse → Chunk → Enrich → Embed → Index

ONLINE (mỗi query): PreRAG → Retrieve → Rerank → Augment → Generate → PostRAG

Không bước nào “an toàn” — mỗi bước đều có thể là nơi pipeline hỏng.

Một AI assistant nội bộ trả lời câu hỏi chính sách. Bạn nhận được 4 case lỗi từ log production.

Nhiệm vụ (5 phút): Với mỗi case **đánh dấu bước** chịu trách nhiệm (**Retrieval / Rerank / Augment / Generate**) và đề xuất 1 fix:

1. User hỏi “nghỉ phép” nhưng tài liệu chỉ ghi “annual leave” → không tìm thấy gì.
2. Top-10 có chunk đúng ở vị trí #8, nhưng LLM chỉ đọc top-3 → trả lời thiếu.
3. Retrieve đúng chunk, nhưng câu trả lời thêm thông tin **không có** trong chunk.
4. Hai chunk mâu thuẫn (policy v2 vs v3), LLM chọn bản cũ.

Mỗi case 1 dòng — Case #: bước → fix.

Ví dụ Case 1: Retrieval → query rewrite / thêm synonym.

PHẦN 02

OFFLINE — Ingestion

Nửa trái bản đồ: chunk → enrich → embed → index. Làm **một lần** lúc index, nhưng **quyết định trần chất lượng** cho mọi query về sau — retrieval không thể tốt hơn cái đã index

BƯỚC HIỆN TẠI

Chunking

Cắt tài liệu sao cho giữ ngữ cảnh

OFFLINE

1 lần



ONLINE

mỗi query

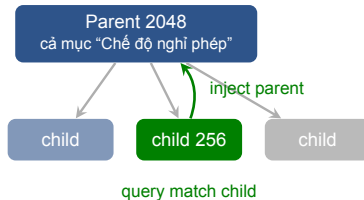


Chiến lược	Tham số mặc định	Khi nào dùng
Fixed-size <i>Day 7</i>	512 tokens, overlap 64	Baseline, text đồng nhất
Semantic <i>Day 7</i>	cosine threshold 0.85	Đoạn đổi chủ đề, chat log
Hierarchical	parent 2048 / child 256	Default doc có cấu trúc
Structure-aware	theo heading/table	Markdown, HTML, PDF
Late chunking	embed doc trước, cắt sau	Tài liệu dài, tham chiếu ngược
RAPTOR	cây tóm tắt đệ quy	Hỏi tổng hợp xuyên đoạn

Hierarchical (default), **Late chunking**, **RAPTOR** (tổng hợp) dạy riêng ở 3 slide kế. Còn lại: tham khảo.

Mâu thuẫn Day 08: chunk *nhỏ* = retrieve chính xác nhưng **thiếu ngữ cảnh**; chunk *lớn* = đủ ngữ cảnh nhưng **nhiều, khó match**.

Cách làm: index **2 tầng** — child 256 (để tìm) trở về parent 2048 (để đọc). Match trên child, nhưng **inject parent** vào LLM.

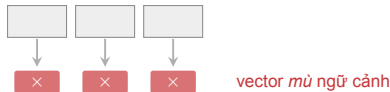


Lưu ý: **Default tốt** cho doc có cấu trúc (policy, manual). **Cách dùng:** LlamaIndex AutoMergingRetriever / LangChain ParentDocumentRetriever — khai báo parent+child size, retriever tự gộp. **Chi phí:** lưu cả 2 tầng.

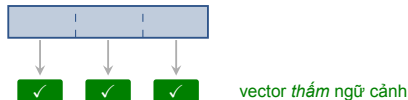
Cách thông thường: cắt chunk *trước* rồi embed từng cái rồi → mỗi chunk **mù** phần còn lại của doc (“điều này”, “công ty” — ai?).

Late chunking: chạy token-embedding cho **cả document trước** (long-context encoder), *rồi mới* pool theo ranh giới chunk. Mỗi chunk vector **đã thăm** ngữ cảnh toàn cục.

Thường: cắt rời → embed riêng



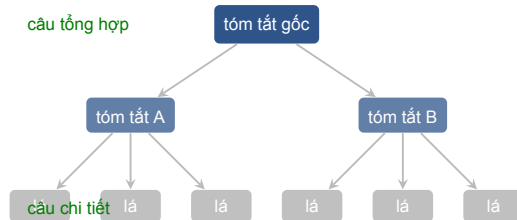
Late: embed cả doc → pool



Lưu ý: **Khi nào:** tài liệu dài, nhiều tham chiếu ngược (“như đã nói ở trên”). **Cách dùng:** cần encoder long-context (Jina-v3, nomic) + thư viện hỗ trợ late chunking. **Khác contextual:** không gọi LLM/chunk — rẻ hơn, nhưng cần model hợp.

Cách thông thường: mọi chunk cùng một mức chi tiết. Câu hỏi tổng hợp (“toàn bộ chính sách phép đổi gì?”) phải gom hàng chục lá → dễ miss, dễ tràn context.

New approach: gom chunk → cụm → LLM tóm tắt, *lắp lên* thành **cây**. Retrieve ở **đúng độ cao**: chi tiết → lá; tổng hợp → nút cao.



Lưu ý: câu hỏi “theme / tổng hợp xuyên nhiều đoạn” mà chunk phải luôn trả thiếu.
Chi phí: build cây = nhiều LLM tóm tắt lúc index, index phình to. (Sarathi et al., ICLR 2024)

BƯỚC HIỆN TẠI

Enrich

Làm chunk giàu thông tin hơn trước khi embed

OFFLINE

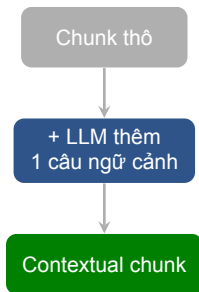
1 lần



ONLINE

mỗi query





“Đoạn này thuộc chính sách nghỉ phép 2026...” ghép vào trước chunk.

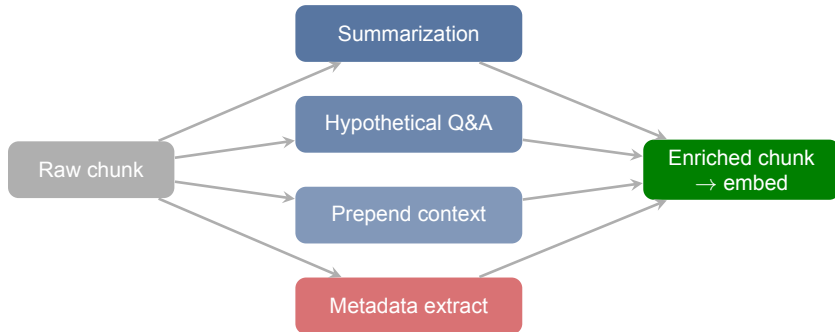
Vấn đề: chunk rời rạc mất ngữ cảnh (“điều này” — điều gì?).

Hiệu quả (Anthropic) — *lỗi retrieval* giảm so baseline:

Cấu hình	Lỗi retrieval
Contextual (embed + BM25)	↓ 49%
+ reranking	↓ 67%

Số liệu Anthropic (2024), tùy corpus. Chi phí: +1 LLM call/chunk lúc index (one-time, model rẻ). Nhồi cả document mỗi call → tốn token; dùng **prompt caching** để giảm. **Bổ sung**, không thay reranking.

Lưu ý: Với corpus văn bản dài / nhiều tham chiếu mơ hồ, đây thường là kỹ thuật **OFFLINE ROI cao** — trả tiền 1 lần lúc index. Đánh đổi: +1 LLM call/chunk lúc index (phải chạy lại mỗi lần re-index — đổi chunking, đổi model); corpus ngắn hoặc đã rõ



- **Summarization** — embed bản tóm tắt cạnh chunk gốc: lọc nhiễu, bám ý chính.
- **Hypothetical Q&A** — LLM sinh câu hỏi mà chunk trả lời được: nối vocab user ↔ tài liệu.
- **Metadata extract** — LLM rút entity, topic, ngày, nguồn → bật metadata filtering.

Tất cả là chi phí one-time lúc index. Dùng model rẻ (GPT-4o-mini, Haiku). Cân nhắc theo giá trị/query.

Vấn đề: user hỏi bằng *câu hỏi*, tài liệu viết bằng *câu khẳng định*. Embedding của câu hỏi và đoạn văn thường **lệch** nhau.

Cách làm: lúc index, LLM sinh **các câu hỏi mà chunk này trả lời được**, embed chúng *cạnh* chunk. Query của user giờ match thẳng vào câu hỏi giả định.

Chunk: "...12 ngày phép/năm..."

LLM sinh Q

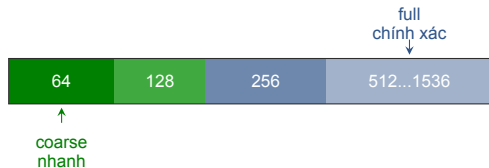
"Nhân viên mới được nghỉ bao nhiêu ngày?"

embed & index cạnh chunk gốc

Lưu ý: **Khi nào:** FAQ, hỏi-đáp chính sách — nơi user dùng từ khác tài liệu. **Cách dùng:** 1 LLM call/chunk lúc index (model rẻ), lưu Q-embeddings trở về chunk. **Họ hàng** HyDE (Phần ONLINE) — nhưng làm *offline*, không tốn call mỗi query.

Cách thông thường: embedding = 1 vector *cố định* (vd 1536-d). Muốn nhanh/rẻ hơn → phải đổi model & re-embed cả corpus.

New approach: train sao cho 1 vector *chứa nhiều vector lồng nhau*. **Cắt ngắn** prefix = nhanh hơn, mất ít chất lượng. Embed 1 lần, dùng nhiều ngân sách.



Dùng 128-d lọc thô (nhanh) → full-d rerank. OpenAI text-embedding-3 hỗ trợ cắt chiều.

Lưu ý: đổi cost/latency *không cần re-embed* — một index phục vụ nhiều SLA. **Chi phí:** cần model train kiểu MRL; cắt càng sâu chất lượng giảm dần. (Kusupati et al., NeurIPS 2022)

Chiến lược	Tốt cho (data)	Tránh khi	Tool chính	Tin hiệu RAGAS
Fixed-size	FAQ, doc ngắn đồng nhất	Doc cấu trúc chặt: cắt giữa ý (legal)	TokenTextSplitter	Context Recall thấp
Sentence Window	KB article, HR policy	Bảng / code: ranh giới câu vô nghĩa	SentenceWindow Node-Parser	Faithfulness cao
Hierarchical	Spec, hợp đồng, ADR	Text phẳng: không có cây để khai thác	Hierarchical NodeParser	Context Precision cao
Semantic	Corpus hỗn hợp, re-search	Corpus lớn: embed từng câu, ingest chậm/đắt	SemanticSplitter Node-Parser	Cân bằng mọi metric
Layout-aware (PDF)	PDF scan, report nhiều bảng	Doc text thuần: thừa, không có layout	PyMuPDF, pdfplumber	Vá recall ở bảng
Multimodal (slide/ảnh)	Deck, doc nhiều diagram	Corpus chỉ text: thừa chi phí vision	LlamaParse, GPT-4V	Context Recall (ảnh)

Bắt đầu từ **loại dữ liệu** của bạn → chọn cột “Tốt cho”. Nếu RAGAS báo metric ở cột cuối tụt → đang dùng sai chiến lược cho data đó.
Default an toàn: Hierarchical cho doc có cấu trúc.

PHẦN 03

ONLINE — Retrieval

Nửa phải bản đồ: retrieve → rerank → control flow. Chạy lại **mỗi query**; mỗi vấn đề trong Error Tree có một fix riêng ở đây — không bật hết, bật đúng cái cần

BƯỚC HIỆN TẠI

Retrieve

Tìm đúng chunk cho mỗi query

OFFLINE

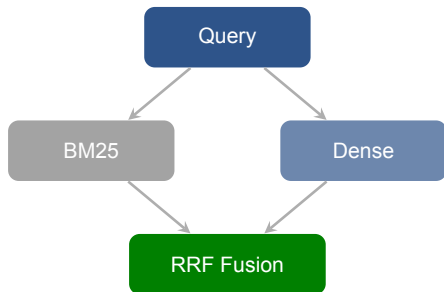
1 lần



ONLINE

mỗi query





	BM25	Dense	Hybrid
Exact keyword	☒	☐	☒
Synonym	☐	☒	☒
Latency	<5ms	~20ms	~25ms

Tiếng Việt: BM25 cần word segmentation (underthesea, VnCoreNLP) trước khi index.

$$\text{score}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)} \quad k=60 \text{ (tinh chỉnh 10-100)}$$

Không cần training, gộp rank từ nhiều nguồn khác thang điểm. Đây là baseline — phần sau là khi RRF **chưa đủ**.

Đoán — Câu trả lời nằm ở 1 token hiếm (mã “X7-204”). RRF gộp rank mức *document*. **Tại sao vẫn miss?** Nghĩ 30s trước khi xem bảng — gợi ý: cần so khớp **sâu hơn document** (mức token).

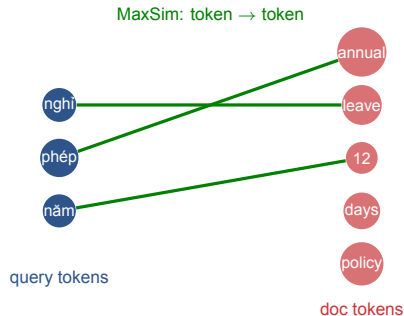
Kỹ thuật	Ý tưởng	Precision	Latency
RRF (baseline)	Gộp rank, không cần train	—	~25ms
Late interaction (Col-BERT)	So khớp token-by-token (MaxSim); tốn storage multi-vector	+8–12%	~50ms
Learned sparse (SPLADE)	Sparse vector học được trên inverted index; storage gần BM25	+6–10%	~40ms
Tensor / weighted fusion	Outer product feature; cần label data	+10–15%	~80ms

Con số tham khảo, tùy

dataset/baseline; đổi precision lấy latency + storage — chọn theo SLA, không theo “mới nhất”. 3 slide sau: từng kỹ thuật

Cách thông thường: bi-encoder ép cả đoạn vào 1 vector $\rightarrow$ chi tiết token (tên riêng, số, thuật ngữ hiếm) bị nén mất. So cosine 2 vector.

New approach: giữ 1 vector / token. Mỗi token query tìm token doc khớp nhất (**MaxSim**), cộng lại. Chi tiết được giữ; doc tính trước nên vẫn nhanh hơn cross-encoder.



Lưu ý: truy vấn cần *chi tiết exact* mà dense bỏ lỡ — không phải trả giá cross-encoder. **Cách dùng:** RAGatouille / Vespa / Qdrant multi-vector; cần index riêng (PLAID). **Chi phí:** nhiều vector/doc (storage lớn). (Khattab & Zaharia, SIGIR 2020)

Cách thông thường: BM25 chỉ match đúng từ xuất hiện. “nghỉ phép” không khớp “annual leave” — vocab gap. Dense bắt synonym nhưng mờ token hiếm.

New approach: model học trọng số từ và tự thêm từ liên quan (term expansion) vào 1 sparse vector. Vẫn chạy trên **inverted index** (nhanh, rẻ như BM25) nhưng thông minh hơn.

Chunk

“annual leave 12 days”

BM25 {annual, leave, 12, days}

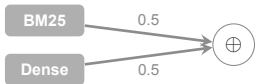
SPLADE {annual, leave, **vacation, holiday, pto**, 12, days}

Term expansion (in xanh) đóng vocab gap lúc index — query “pto” vẫn tìm thấy.

Lưu ý: vocab gap + cần tốc độ inverted index (không lưu multi-vector như ColBERT).

Cách dùng: `naver/splade` encoder → Qdrant/Elasticsearch sparse vector. **Chi phí:** cần GPU lúc index để sinh sparse vector. (Formal et al., SIGIR 2021)

RRF: cân bằng cứng 0.5 / 0.5



Fusion: học 0.3 / 0.7

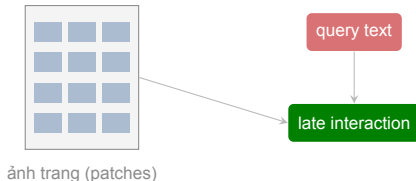


“Label” là gì? — Cặp (query → doc nào đúng). Nguồn: người gán nhãn relevance (vàng, đắt) hoặc click/feedback log (weak label, rẻ). Có label → train fusion; không label → RRF.

Lưu ý: Cách dùng: thu click-log trước, train fusion sau — đừng bắt đầu từ đây. Mặc định vẫn RRF tới khi đủ label. Đợi lấy 10–15% precision.

Cách thông thường: PDF → OCR/parse → text → chunk → embed. Parsing *giòn*: bảng, layout, scan, biểu đồ mất sạch. “Garbage in”.

New approach: vision-LLM embed *thẳng ảnh trang* (patch embeddings) + late interaction với query. **Bỏ qua cả pipeline parse** — bảng/biểu/layout giữ nguyên.



Lưu ý: tài liệu nặng layout/bảng/scan (tài chính, form, hợp đồng) — nơi parse *luôn* hỏng. **Cách dùng:** ColPali/ColQwen + Vespa/Qdrant multi-vector. **Chi phí:** lưu patch embeddings (lớn), cần vision model, tooling còn non. (Faysse et al., 2024)

Hỏi **bạn cần gì**, theo thứ tự — dừng ở dòng đầu tiên khớp:

Đã có Postgres, dữ liệu vừa phải? —————>

pgvector

Muốn managed, zero-ops? —————>

Pinecone

Cần hybrid + metadata filter (đa số ca)? —————>

Qdrant *(default)*

Cần quan hệ giữa nhiều entity? —————>

Neo4j *(GraphRAG, Day 19)*

Khác: Weaviate (thích GraphQL) · Milvus

(quy mô lớn, GPU) · Chroma/FAISS (prototype). Tất cả đều làm vector search — khác nhau ở hybrid, filter, scale, ops.

Lưu ý: Lab 18 dùng **Qdrant** (Docker local): hybrid + metadata built-in, đủ cho pipeline production end-to-end.

Kỹ thuật	Tốt cho (data)	Chi phí / Tránh khi	Tool chính	Tín hiệu RAGAS
Hybrid + RRF	Đa số ca: exact + synonym	Không học trọng số	Qdrant hybrid	Baseline (~25ms)
ColBERT <i>late interaction</i> SPLADE <i>learned sparse</i>	Cần precision token-level Vocab mismatch, domain hẹp	Storage +8–12% Cần train/encode	RAGatouille, Vespa SPLADE + inverted idx	Context Precision cao Context Recall ↑
Weighted Fusion <i>tensor</i>	Có labeled data để tune	Cần label để học trọng số	Vespa rank profile	Precision ổn định ↑
ColPali <i>visual</i>	PDF/slide không parse được	GPU đắt; thừa nếu corpus text	ColPali, byaldi	Context Recall (ảnh)

Mặc định Hybrid + RRF, chỉ leo lên khi RAGAS chỉ rõ điểm đau: precision tụt → ColBERT; recall do vocab → SPLADE; có label → Fusion; ảnh/PDF → ColPali. **Đừng bật hết** — mỗi cái thêm latency/storage.

BƯỚC HIỆN TẠI

Rerank

Sắp xếp lại để chunk đúng lên top

OFFLINE

1 lần



ONLINE

mỗi query





Retriever (bi-encoder) embed query & doc **riêng** → nhanh nhưng thô. Reranker (cross-encoder) đọc **cặp query+doc cùng lúc** → chính xác nhưng đắt. Lấy rộng rồi lọc tinh.

Lưu ý: ROI: tốn ~30–50ms/query, đổi lấy **+15–25% precision** (số tham khảo, tùy data). Khi latency cho phép, đây thường là một trong những tối ưu lời nhất.

TaiLieuBot: top-20 lần cả “nghỉ phép / nghỉ ốm / nghỉ thai sản” → reranker đẩy đúng mục “nghỉ phép” lên top-3.

Model	Chi phí	Khi nào dùng
Qwen3-Reranker	Open-weight	Top MTEB-rerank, đa ngôn ngữ, 0.6B–8B tùy budget
Cohere Rerank v3.5	API (\$)	Lựa chọn managed phổ biến, đa ngôn ngữ
bge-reranker-v2-m3	Open-weight	Tiếng Việt tốt, 8K context, self-host
Jina Reranker v2	Open-weight	Đa ngôn ngữ, 8K context
ms-marco-MiniLM-L-12	Open-weight	Nhẹ, nhanh, thiên English
Flashrank	Open-weight	Siêu nhẹ, <5ms, edge
LLM-as-Reranker	\$\$\$	GPT-4/Claude, chất lượng cao — chậm & đắt

Open-weight = không phí

license, nhưng vẫn tốn compute để self-host — không đáng kể nếu model nhỏ.

Chọn đi — TaiLieuBot: **tiếng Việt, self-host, latency cho phép.** Reranker nào? Chọn 1

BƯỚC HIỆN TẠI

Control Flow

Định hình query & làm sạch context theo vấn đề

OFFLINE

1 lần



ONLINE

mỗi query



Vấn đề	Tầng	Pattern
Query lệch vocab so với corpus	PreRAG	HyDE
Câu hỏi nhiều phần / multi-hop	PreRAG	Multi-Query
Retrieve chất lượng thấp, cần fallback	PreRAG	Corrective RAG
Context dư thừa / mâu thuẫn	Augment	NLI filter, compression
Cần kiểm chứng nguồn	Augment	Citation injection

Lưu ý: Mỗi pattern sửa **một vấn đề** — các slide sau là chi tiết PreRAG / Augment (cache, reliability để dành Day 25).

HyDE nhắc Day 8

LLM sinh **câu trả lời giả định** → embed câu đó để tìm.

Nổi vocab gap (“nghỉ phép” ↔ “annual leave”). Họ hàng Hypothetical Q&A (OFFLINE) — nhưng làm **mỗi query**.

Multi-Query nhắc Day 8

Tách query phức → nhiều **sub-query**, retrieve rồi gộp.

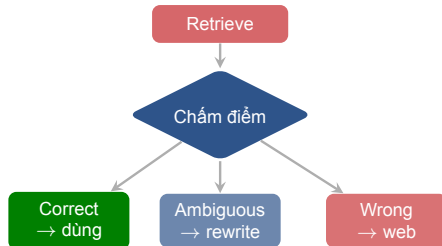
“Phép khác ốm thế nào?” → 2 sub-query rồi hợp kết quả.

Lưu ý: Điều **tốn +1 LLM call mỗi query** → bật theo vấn đề (HyDE: vocab gap; Multi-Query: nhiều phần), không mặc định. Corrective RAG (fallback khi retrieve kém) → slide kế.

*Khác OFFLINE (trả 1 lần cho cả corpus): mỗi pattern ở đây = +1 LLM call **mỗi query**. TaiLieuBot: “nghỉ phép” bật HyDE; “phép khác ốm?” bật Multi-Query.*

Cách thông thường: retrieve được gì thì nhồi thẳng vào LLM — kể cả khi chunk *lạc đề*. Rác vào → bịa ra.

CRAG: 1 evaluator **chấm điểm** chunk retrieve được. Theo điểm → rẽ nhánh: dùng luôn / viết lại query / **fallback web search**.



Lưu ý: Khi nào: corpus có lỗi hổng, câu hỏi ngoài phạm vi — thà fallback còn hơn bịa.
Cách dùng: LangGraph node “grade → route”; evaluator = LLM rẻ hoặc classifier. **Chi phí:** +1 bước chấm điểm/query. Họ hàng Self-RAG (→ Day 23). (Yan et al., 2024)

Tầng giữa Retrieve và Generate — Top-k chunks hiếm khi sạch: dư thừa, mâu thuẫn, thiếu nguồn. Augment chèn bước **làm sạch** trước khi đưa vào LLM — bắt **đúng bước** theo vấn đề (giống PreRAG), không bắt cả 4.

Bước	Vấn đề	Cách làm	Tool
NLI verification	Chunk lạc đề / mâu thuẫn câu hỏi	Kiểm entailment query $\leftrightarrow$ chunk, bỏ chunk “contradiction”	nli-deberta-v3
Compression	Prompt quá dài, nhiều nhiễu	Nén / bỏ câu ít liên quan, giữ ý chính	LLMLingua
Citation injection	Cần kiểm chứng nguồn	Gắn [1][2] mỗi câu, trace ngược về chunk gốc	prompt + map
Conflict resolution	2 chunk mâu thuẫn (v2 vs v3)	newest-wins (theo metadata) / LLM làm trọng tài	metadata + LLM

Lưu ý: Nhồi top-k thẳng vào prompt (như demo) **bỏ lỡ tầng này**. TaiLieuBot case #4: policy v2 vs v3 mâu thuẫn → conflict resolution chọn v3 (newest-wins).

PHẦN 04

Close the Loop & Limits

Đo bằng RAGAS → tra ngược về Error Tree (Phần I) để sửa
đúng tầng — vòng lặp khép kín; rồi nhìn thẳng giới hạn embed-
ding và ranh giới với các ngày khác

BƯỚC HIỆN TẠI

Close the Loop

Đo bằng RAGAS, tra Error Tree, sửa đúng tầng

OFFLINE

1 lần



ONLINE

mỗi query



RETRIEVAL — lấy đúng chưa?

Context Recall ≥ 0.75

Đã lấy đủ thông tin cần thiết?

Context Precision ≥ 0.75

Chunk liên quan có xếp *hạng cao* top-k?

Thấp → sửa tầng Retrieve/Rerank.

GENERATION — trả lời đúng chưa?

Faithfulness ≥ 0.85

Câu trả lời có *bám* context (chống bịa)?

Answer Relevancy ≥ 0.80

Trả lời đúng *trọng tâm* câu hỏi?

Thấp → sửa tầng Generate (prompt).

Một điểm tổng *che* chỗ hỏng. Tách retrieval vs generation → biết ngay **tầng nào** mất điểm để tra Error Tree. (RAGAS sâu, LLM-as-Judge, guardrails → Day 24.)

Cùng bản đồ pipeline ở **Phần 01** — giờ vào từ cột **metric** thay vì vấn đề. RAGAS chỉ ra tăng yếu → tra về cùng la bàn → sửa → đo lại.

Metric thấp	Hành động (tăng nào trong bản đồ)
Context Recall	Hybrid (+BM25), HyDE, Multi-Query, metadata filter (Retrieval)
Context Precision	Reranking, tăng threshold, metadata filter (Rerank)
Faithfulness	Prompt chặt, hạ temperature, citation (Generate)
Answer Relevancy	Cải thiện prompt, mở rộng context (Generate)

(1) RAGAS baseline → (2) bottom-5 fail → (3) đọc metric thấp → tra bảng = bản đồ Phần I → (4) sửa → (5) so delta. **Thứ tự rõ → đắt:** recall (hybrid) → precision (rerank) → faithfulness (prompt).

Negation insensitivity

“được nghỉ” vs “**không** được nghỉ” → cosine cao nhưng nghĩa ngược.

Temporal blindness

Vector không mã hóa thời gian → chính sách **2024 vs 2026** cùng score.

Entity-swap

“phép **nhân viên**” vs “phép **quản lý**” → vector gần nhau, dễ nhầm.

Numerical blindness

“nghỉ **12** ngày” vs “nghỉ **20** ngày” → cosine cao, nhưng con số mới là đáp án.

Lưu ý: 4 ca này đều **cosine cao nhưng nghĩa khác** — embedding một mình không phân biệt được phủ định / thời gian / thực thể / con số. Không phải “kém”, mà **chưa đủ**. Cách bù đắp → slide kế.

- **Metadata filtering** (temporal, số, source) — sửa temporal & numerical blindness: lọc theo ngày / con số trước khi search.
- **NLI verification** post-retrieval — bắt negation/entity-swap: kiểm entailment query $\leftrightarrow$ chunk.
- **Failure-aware routing** — phát hiện loại lỗi trước generate, đổi chiến lược (rewrite / decompose / từ chối).
- **GraphRAG** — câu hỏi quan hệ giữa nhiều entity: graph thay vì flat vector ($\rightarrow$ **Day 19**).
- **Human-in-the-loop** — high-stakes: người duyệt trước khi trả ($\rightarrow$ **Day 27**).

Không phải cứ thêm data hay đổi embedding model là fix được. **Chẩn đoán loại lỗi trước**, rồi chọn đúng công cụ.

Chất lượng **ingestion (OFFLINE)** + **retrieval (ONLINE)**: chunking, contextual, enrichment, hybrid/rerank, query transform, augmentation — thường là phần “ăn điểm” lớn của RAG.

Chủ đề liên quan	Học sâu ở đâu
GraphRAG & Knowledge Graphs	Day 19
Multi-Agent / Agentic RAG patterns	Day 20 (Multi-Agent), Day 23 (Lang-Graph)
RAGAS sâu, LLM-as-Judge, Guardrails	Day 24
Caching, circuit breakers, reliability	Day 25
Human-in-the-loop UX	Day 27

Hôm nay chỉ dùng

các chủ đề này ở mức “đủ để pipeline chạy & đo được”; chi tiết để dành đúng ngày.

Hands-on 18

Đủ lý thuyết — giờ tự tay dựng pipeline trên dữ liệu thật, đo bằng RAGAS, và phân tích thất bại như TaiLieuBot

01

Cá nhân — 2h implement + 30 phút reflection

Implement **tất cả 5 module** (TODO markers trong code):

1. `m1_chunking.py` — semantic, hierarchical, structure-aware
2. `m2_search.py` — BM25 Vietnamese + Dense + RRF
3. `m3_rerank.py` — CrossEncoder rerank top-20 → top-3
4. `m4_eval.py` — RAGAS 4 metrics + failure analysis
5. `m5_enrichment.py` — contextual / summary / hypo-Q&A

Verify: `pytest tests/test_m*.py` · Pipeline: `python src/pipeline.py`

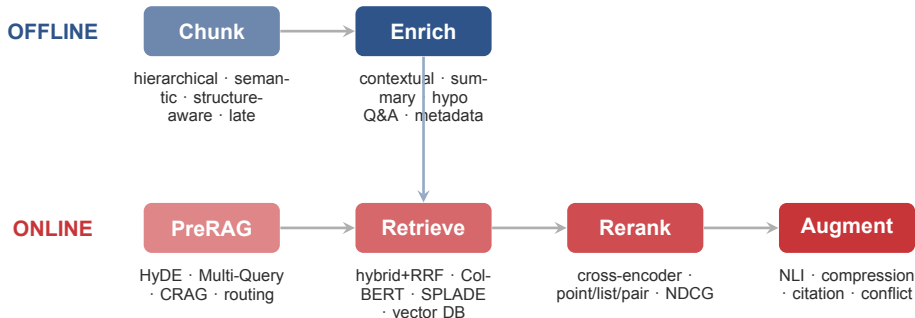
Reflection (30 phút): map lecture concept → code vừa viết · khó khăn & cách giải · action plan cho project cá nhân

```
git clone .../lab18 → docker compose up -d (Qdrant) → pip install -r requirements.txt → python naive_baseline.py (lấy điểm gốc để so)
```

Pipeline	Cấu hình	Kỳ vọng RAGAS (minh hoạ)
A — Naive	paragraph chunk 512 + dense-only	Faithfulness ~ 0.70 , Recall ~ 0.55
B — Production	hierarchical + hybrid + bge-reranker	Faithfulness ≥ 0.85 , Recall ≥ 0.75
C — Tự chọn	B + 1 hoặc nhiều kỹ thuật nâng cao (tự chọn)	đo delta vs B

Thêm vào B **một hoặc nhiều** kỹ thuật rồi đo cải thiện. Gợi ý: contextual embeddings · summary/hypothetical-Q&A enrich · metadata filter · HyDE · Multi-Query · semantic chunk · threshold rerank. Không giới hạn ở contextual.

Lưu ý: Cùng dữ liệu, cùng judge model — chênh lệch điểm đến **phần lớn** từ kỹ thuật pipeline. Con số minh hoạ; số thật đo trong lab.



RAG yếu trong production → (1) đo RAGAS biết **retrieval** hay **generation** hỏng → (2) Error Tree map vấn đề ra **tầng** → (3) bản đồ này chọn **nút vặn**. Đừng đổi model, vặn đúng tầng. (Đo→Day24 · embedding mù→GraphRAG Day19 / HITL Day27.)

Key Takeaways

Production RAG là kỹ thuật chẩn đoán + sửa đúng tầng, không phải thêm model mạnh hơn

02

1

Chẩn đoán trước, sửa sau. RAG = OFFLINE + ONLINE; dùng Error Tree để biết tầng nào mất điểm.

2

OFFLINE đặt trần chất lượng. Hierarchical chunk + contextual + enrichment: làm 1 lần lúc index, lợi mọi query — retrieval không thể tốt hơn cái đã index.

3

Reranking ROI cao khi recall đã đủ. +15–25% precision với ~30–50ms *khi latency cho phép*; vô dụng nếu chunk đúng chưa được retrieve. Chọn theo ngôn ngữ & chi phí.

4

Đo để sửa đúng chỗ. RAGAS 4 metric → diagnostic tree → sửa tầng yếu (eval sâu: Day 24).

5

Biết giới hạn. Embedding mù negation/temporal — chẩn đoán loại lỗi trước khi thêm data.

1. Anthropic. *Introducing Contextual Retrieval* (2024).
<https://www.anthropic.com/news/contextual-retrieval>
2. Weaviate. *Advanced RAG Techniques* ebook.
3. ES Khattab & Zaharia. *ColBERT: Late Interaction*. SIGIR 2020.
4. Sarthi et al. *RAPTOR: Recursive Abstractive Tree*. ICLR 2024.
5. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS 2022.
6. Faysse et al. *ColPali: Vision-Language Retrieval*. arXiv:2407.01449 (2024).
7. Gao et al. *Precise Zero-Shot Dense Retrieval (HyDE)*. arXiv:2212.10496
8. Asai et al. *Self-RAG*. arXiv:2310.11511 (đọc thêm → Day 23)
9. Yan et al. *Corrective RAG (CRAG)*. arXiv:2401.15884
10. Es et al. *RAGAS*. arXiv:2309.15217
11. Ehtesham et al. *Agentic RAG Survey*. arXiv:2501.09136 (đọc thêm → Day 20/23)

Day 19 — GraphRAG & Knowledge Graphs

“Khi user hỏi về quan hệ giữa 5 entity, flat RAG trả lời sai — GraphRAG trả lời đúng.”

- Hoàn thiện Lab 18: pipeline + RAGAS report đạt ≥ 0.75
- Thử 1 cải tiến M5 khác, đo delta vs baseline
- Đọc Microsoft GraphRAG paper (2024) trước Day 19

Hỏi & Đáp

*Production RAG: bạn sẽ sửa tầng nào
đầu tiên trong hệ thống của mình?*



Cảm ơn!

Trần Quang Thiện · AICB Phase 2 · Track 3

Production RAG — Ngày 18